

Modul 13 Perintah Dasar Linux

Tujuan Praktikum

1. Praktikan menguasai perintah-perintah dasar Linux.
2. Praktikan dapat meng-*compile* program dengan gcc.

13.1 Perintah-Perintah Dasar Linux

Pada umumnya *default* perintah dasar Linux dieksekusi pada *Bash* (`/bin/bash`). *Bash* mengeksekusi program dalam OS untuk setiap perintah yang dimasukkan dalam *console*. Berikut adalah beberapa petunjuk dalam menulis perintah:

- Tekan *enter*. Tidak ada yang terjadi hingga anda menekan *enter*.
- Gunakan *Tab* untuk *auto completion*. Biasakan gunakan *tab*.
- Ctrl+C untuk membatalkan perintah. Ctrl+C memberikan *attempt signal* ke program.
- Shift + *PageUP* untuk melihat hasil *text* sebelumnya .
- Panah atas/bawah untuk mengulang perintah.

Perintah–perintah di *console* Linux memiliki aturan–aturan penulisan, aturan struktur penulisan perintah Linux tersebut berlaku pada hampir semua perintah–perintah Linux. Penulisan perintah Linux bersifat *case sensitive* artinya adalah setiap penulisan huruf besar dan kecil sangat berpengaruh terhadap hasil yang diinginkan. Berikut struktur penulisan perintah Linux:

```
$ Perintah [-opsi] [argument_1] [argument_2]
```

Tabel 13-1 Penjelasan Struktur Perintah Linux

Nama bagian	Keterangan
\$	Merupakan tanda representasi <i>user mode</i> . Tanda representasi tersebut di bagi menjadi 2 yaitu \$ dan #, berikut adalah penjelasan dari tanda <i>user mode</i> tersebut: • # (biasa disebut dengan <i>root</i>) adalah <i>user mode</i> yang memiliki hak akses tertinggi, <i>root</i> memiliki hak akses yang tak terbatas terhadap seluruh perintah Linux. Perintah – perintah yang dapat di jalan pada <i>mode root</i> terdapat di <i>folder /etc/sbin</i>. • \$ adalah tanda untuk <i>user mode</i> biasa yang hanya memiliki hak akses terbatas terhadap beberapa perintah – perintah Linux. Perintah – perintah yang dapat di jalan kan ketika <i>user</i> menggunakan <i>mode \$</i> disimpan di dalam <i>folder /etc/bin</i>.
Perintah	Perintah adalah program yang digunakan untuk berinteraksi dengan <i>system</i> operasi Linux.
[-opsi]	Opsi adalah fungsionalitas – fungsionalitas <i>default</i> yang dimiliki oleh perintah yang digunakan. Kita dapat mengkombinasikan beberapa opsi untuk mendapatkan hasil yang diinginkan. Fungsionalitas dari setiap perintah Linux dapat di baca dengan menggunakan perintah \$ man perintah
[argument_1]	<i>Argument_1</i> , biasanya adalah nama <i>file</i> , direktori atau alamat (<i>relative</i> atau <i>absolute</i>) dari suatu <i>file</i> ataupun <i>folder</i> .
[argument_2]	<i>Argument_2</i> , pada beberapa perintah Linux <i>argument_2</i> bersifat <i>optional</i> dengan kata lain tidak selalu harus ada, namun untuk beberapa perintah operasi <i>argument_2</i> biasanya

berisi nama, direktori atau alamat (<i>relative</i> atau <i>absolute</i>) dari suatu <i>file</i> ataupun <i>folder</i> tujuan

13.1.1 ls (List Dictionary)

\$ **ls** adalah perintah yang digunakan untuk melihat isi dari sebuah *directory* atau *file* di direktori aktif. Terdapat beberapa opsi fungsionalitas *default* yang dimiliki oleh perintah \$ **ls**, berikut adalah contoh penggunaan perintah \$ **ls**:

```
$ ls [opsi] [directory]
$ ls -al /home/praktikan
```

Berikut beberapa opsi yang dapat digunakan pada perintah \$ **ls** :

Tabel 13-2 Sebagian Daftar Opsi Perintah ls

Opsi	Keterangan
-l	menampilkan tampilan secara detail dari setiap <i>file</i> yang di tampilkan.
-a	menampilkan seluruh <i>file</i> yang terdapat di dalam <i>directory</i> termasuk <i>hidden file</i>
-s	menampilkan ukuran <i>file</i> (<i>in blocks, not bytes</i>)
-h	menampilkan kedalam bentuk " <i>human readable format</i> " (ie: 4K, 16M, 1G etc).

13.1.2 cd (Change Dictionary)

\$ **cd** adalah perintah yang digunakan untuk mengubah posisi dari posisi *directory* kerja aktif ke *directory* kerja yang akan kita gunakan. Contoh:

```
$ cd /usr/local
$ pwd /usr/local
```

Perintah diatas di gunakan untuk berpindah dari *directory* yang kita tempati sekarang menuju ke *directory* /usr/local

Selain itu perintah cd dapat digunakan untuk berpindah ke *parent directory* dari suatu *directory* aktif, yaitu dengan menambahkan (..) setelah perintah \$ **cd**.

Contoh:

```
$ pwd /usr/local
$ cd ..
$ pwd /usr
```

13.1.3 cp (Copy)

\$ **cp** adalah perintah untuk meng-*copy* satu atau banyak *file* dalam satu kali penulisan perintah atau duplikasi *file* atau *folder* dari sumber ke tujuan. Perintah yang di gunakan untuk meng-*copy file* atau *folder* adalah

```
$cp [opsi] /folder_asal/nama_file /folder_tujuan
```

Tabel 13-3 Sebagian Daftar Opsi Perintah cp

Opsi	Keterangan
-R	Digunakan untuk meng- <i>copy</i> suatu <i>directory</i> beserta dengan isinya
-v	Digunakan untuk melihat <i>file</i> yang di- <i>copy</i> -kan ke tujuan di <i>console</i>

Contoh:

```
$ cp -vR /home/student /data/backup
```

Keterangan: Perintah diatas akan meng-*copy* direktori *students* yang berada dibawah direktori */home* beserta seluruh isinya kedalam direktori */data/backup* dan menampilkan seluruh *file* yang di *copy* kan.

13.1.4 rm (Remove)

\$ **rm** adalah perintah yang digunakan untuk penghapusan (*delete*) satu atau banyak *directory* atau *file* melalui *console*. Untuk menghapus *file* digunakan perintah seperti dibawah ini:

```
$ rm [opsi] nama_file/folder
```

Beberapa opsi yang dapat digunakan dalam perintah **rm**:

Tabel 13-4 Sebagian Daftar Opsi Perintah rm

Opsi	Keterangan
-f	Memaksa <i>file</i> untuk di hapus, biasa di gunakan untuk menghapus <i>file system</i> .
-i	menampilkan peringatan sebelum proses penghapusan di lakukan
-r	Penghapusan secara berulang hingga akhir <i>directory</i> . HATI-HATI apabila menggunakan opsi ini!!!

Contoh:

```
$ rm -f UTS.java
```

Keterangan: Menghapus secara paksa *file* UTS.java

```
$ rm -rf /opt/test1
```

Keterangan: Menghapus seluruh *file* yang terdapat di dalam *folder test1*, dalam hal ini *test1* adalah *folder* yang di dalam nya terdapat beberapa *file*.

13.1.5 mv (Move)

\$ **mv** adalah perintah yang digunakan untuk melakukan perpindahan *file* atau direktori atau dapat juga digunakan untuk melakukan perubahan nama *file* atau direktori jika sumber dan tujuan yang diberikan terletak dalam satu struktur direktori yang sama. Ada beberapa opsi yang dimiliki perintah \$ **mv** ini, berikut adalah opsi yang dapat digunakan pada perintah \$ **mv**

Tabel 13-5 Sebagian Daftar Opsi Perintah mv

Opsi	Keterangan
-R	Digunakan untuk meng- <i>copy</i> suatu <i>directory</i> beserta dengan isinya
-v	Digunakan untuk melihat <i>file</i> yang di- <i>copy</i> -kan ke tujuan

Contoh:

```
$ mv contoh1.html /home/student/contoh2.html
```

Keterangan: Perintah di atas digunakan untuk mengganti nama *file* *contoh1.html* menjadi *contoh2.html* dan memindahkan *file* tersebut kedalam *folder* *student* yang berada di dalam *folder* *home*.

13.1.6 mkdir (Make Dictionary)

\$ **mkdir** adalah perintah yang digunakan untuk membuat satu atau beberapa *directory* melalui konsol. Sama hal nya dengan perintah Linux lainnya pada perintah \$ **mkdir** terdapat beberapa opsi yang dapat digunakan, berikut adalah daftar opsi yang dapat digunakan:

Tabel 13-6 Sebagian Daftar Opsi Perintah mkdir

Opsi	Keterangan
-p	Opsi untuk membuat <i>directory</i> secara bertingkat dengan hanya menggunakan satu buah <i>command</i> .
-m	Opsi untuk memberikan <i>permissions</i> dari <i>directory</i> yang kita buat, dengan menggunakan bilangan oktal (akan dijelaskan pada sub bab <i>file permissions</i>).

mkdir memiliki *syntax* penulisan:

```
$ mkdir [opsi] dir_name
```

dir_name adalah nama dari sebuah direktory yang akan dibuat oleh *user*.

```
$ mkdir -p /tmp/{a/{b,c},opt/test1/test2,var/coba}
```

13.1.7 pwd (Print Working Directory)

\$ **pwd** adalah perintah yang digunakan untuk melihat tempat *directory* aktif atau alamat *directory* yang sedang kita tempati saat ini. Contoh:

```
$ pwd /home/praktikan
```

Hasil dari perintah di atas ditunjukan setelah perintah tersebut di jalankan, dan saat ini *directory* kerja aktif nya sedang berada di **/home/praktikan**.

13.1.8 man (Manual)

\$ **man** adalah perintah yang digunakan untuk melihat manual dari suatu perintah Contoh:

```
$ man ls
```

Perintah di atas untuk mengetahui fungsi dari perintah *ls* dan opsi-opsi yang ada dalam perintah *ls*.

13.1.9 | (Pipeline)

Pipeline adalah perintah di Linux yang memungkinkan Anda menggunakan dua atau lebih perintah sedemikian *output* dari satu perintah berfungsi sebagai masukan ke yang berikutnya. Singkatnya, *output* dari setiap proses langsung sebagai masukan ke yang berikutnya seperti pipa. Simbol '|' menandakan pipa. Pipa membantu Anda menggabungkan dua atau lebih perintah pada waktu yang sama dan menjalankan mereka berturut-turut. Contoh:

```
$ cat README.txt | grep rules
```

Terdapat 2 perintah yaitu "*cat*" dan "*grep*". Perintah "*cat*" berfungsi untuk menampilkan isi *file* ke konsol. Perintah "*grep*" berguna untuk mem-filter, *grep rules* berarti akan menampilkan baris dengan kata "*rules*" saja. Pipa (|) berguna untuk menggabungkan 2 perintah tersebut. *Output* dari perintah "*cat*" digunakan sebagai *input* dari perintah "*grep*".

13.1.10 Redirection

Fasilitas *redirection* memungkinkan kita untuk dapat menyimpan *output* dari sebuah proses untuk disimpan ke *file* lain (*Output Redirection*) atau sebaliknya menggunakan isi dari *file* sebagai *input* dalam suatu proses (*Input redirection*). Komponen-komponen dari *redirection* adalah `<`, `>`, `<<`, `>>`.

`>`: menyimpan hasil ke *file* (*overwrite*)

Contoh:

```
$ ls -al > result.txt
```

`>>`: menyimpan hasil ke *file* (*append*)

Contoh:

```
$ cat README >> result.txt
$ cat INSTALL.txt >> result.txt
```

`<`: *file* sebagai *input* proses

`<<`: *file* sebagai *input* proses

13.2 GCC

GCC dikembangkan oleh Richard Stallman, pendiri dari proyek GNU. Richard Stallman mendirikan proyek GNU pada tahun 1984 untuk menciptakan sistem operasi Unix-like lengkap sebagai perangkat lunak bebas (*open*), untuk mempromosikan kebebasan (*free*) dan kerjasama antara komputer pengguna dan pemrogram. GCC ("GNU C Compiler" atau "GNU Compiler Collection") telah berkembang seiring dengan waktu untuk mendukung banyak bahasa C (gcc), C++ (g++), Java (gcj), Fortran (gfortran), Ada (Agas), Go (gccgo), OpenMP, Cilk Plus, dan OpenAcc.

GCC adalah komponen kunci dari apa yang disebut "GNU Toolchain", untuk mengembangkan aplikasi dan menulis sistem operasi. GCC *portable* dan berjalan di banyak *platform*. GCC (dan GNU Toolchain) saat ini tersedia pada semua Unix/Linux. GCC juga diporting ke Windows (oleh Cygwin, MinGW dan MinGW-W64). GCC adalah juga sebuah *cross-compiler* sehingga dapat memproduksi *executable* pada *platform* yang berbeda. Situs utama untuk GCC adalah <http://gcc.gnu.org/>. Versi saat ini adalah 7.3 GCC, dirilis pada tahun 25-01-2018.

13.2.1 Instalasi GCC

Instalasi pada Linux

Buka Terminal, dan masukkan "*gcc--version*". Jika gcc tidak diinstal, sistem akan meminta Anda untuk menginstal gcc.

```
$ gcc --version
```

Cara *install* gcc pada Linux:

```
$ sudo apt-get update
$ sudo apt-get upgrade
$ sudo apt-get install build-essential
```

atau

```
$ sudo apt-get gcc
```

Instalasi pada Windows

Untuk Windows, Anda juga dapat meng-*install* Cygwin GCC, MinGW GCC atau MinGW-W64 GCC.

- Cygwin GCC: Cygwin adalah lingkungan berbasis Unix dan antarmuka baris perintah untuk Microsoft Windows. Cygwin sangat besar dan mencakup sebagian besar Unix alat dan utilitas. Ini juga termasuk umum digunakan *Bash shell*.
- MinGW: MinGW (minimalis GNU for Windows) adalah pelabuhan *GNU Compiler Collection* (GCC) dan GNU Binutils untuk digunakan dalam Windows. Ini juga termasuk MSYS (*Minimal System*), yang pada dasarnya *Bourne shell* (*bash*).
- MinGW-W64: sebuah *fork* dari MinGW yang mendukung 32-bit dan 64-bit Windows. MinGW-W64 (sebuah *fork* dari MinGW, tersedia di <http://mingw-w64.org/doku.php>) mendukung sasaran 32-bit dan 64-bit Windows asli. Anda dapat menginstal "MinGW-W64" di bawah "Cygwin" dengan memilih paket ini (di bawah "devel" kategori):
- mingw64-x86_64-gcc-core: 64-bit C *compiler* untuk target asli 64-bit Windows. *Executable* adalah "x86_64-w64-mingw32-gcc".
- mingw64-x86_64-gcc-g ++: *compiler* C++ 64-bit untuk target asli 64-bit Windows. *Executable* adalah "x86_64-w64-mingw32-g++".
- mingw64-i686-gcc-core: 64-bit C *compiler* untuk target asli 32-bit Windows. *Executable* adalah "i686-w64-mingw32-gcc".
- mingw64-i686-gcc-g ++: *compiler* C++ 64-bit untuk target asli 32-bit Windows. *Executable* adalah "i686-w64-mingw32-g++".

Instalasi pada Ubuntu

- Pastikan package Ubuntu ter-update, gunakan command: **sudo apt update**
- Install package *build-essential*: **sudo apt install build-essential**
- Cek apakah GCC sudah terinstall pada Ubuntu: **gcc --version**

13.2.2 Meng-Compile Program Menggunakan GCC

Cara meng-*compile source code* menjadi program adalah sebagai berikut:

Pada *folder* dengan *source code* jalankan perintah ini:

```
$ gcc source_code.c -o program_hasil
```

atau

```
$ gcc source_code.c
```

Apabila tidak diberikan opsi hasil maka gcc akan menghasilkan program dengan nama: **a.out**

Cara menjalankan program yang telah *dcompile*:

```
$ chmod +x ./program_hasil  
$ ./program_hasil
```